Infraestrutura de software
Davi Maltez Costa Vasconcelos

Relatório Implementação 5

1. Objetivo

O objetivo deste trabalho é implementar um sistema de sincronização e controle de embarque para uma estação de metrô automatizada da METROREC usando threads e primitivas POSIX (pthreads em C), onde o sistema deve garantir de forma thread-safe que os passageiros aguardem e embarquem sem *busy-waiting*, e que o vagão só parta quando estiver satisfatoriamente preenchido (lotado ou com a fila da estação zerada) e com todos os passageiros devidamente sentados

2. Checklist de Requisitos

Um requisito por linha. Sem parágrafo, se precisar explicar algo, use a coluna "como testar".

Requisito do enunciado	Status	Como testar / evidência
README + Makefile	OK	
Relatório	OK	
Implementação	OK	make + ./sync_car

3. Como Reproduzir

Compilação: make

Rodando: ./sync_car

Limpeza de arquivos: make clean

4. Arquitetura

O projeto foi modularizado em sync_car.h (contendo a estrutura struct station e os protótipos das funções), sync_car.c (com toda a lógica de concorrência e sincronização) e sync_car_runner.c (arquivo de testes fornecido contendo a função main).

Decisões Técnicas:

Decisões	Motivo	Consequência
`pthread_mutex_t` na `struct station`	Cumprir a restrição do enunciado e garantir a atomicidade total sobre o estado da estação	Simplificou o controle de concorrência, mas exigiu cuidado rigoroso na ordem de trava/destrava para evitar *deadlocks*

Decisões	Motivo	Consequência
Separação em duas variáveis de condição (`cond_passageiros` e `cond_vagao`)	Criar salas de espera distintas para passageiros aguardando trem e para o trem aguardando o fim do embarque	Eliminou o busy-wait e permitiu acordar exatamente o tipo de thread correto (`signal` vs `broadcast`)
Adição do contador `passageiros_embarcando`	Rastrear quem já saiu da fila da plataforma, mas ainda não chamou `station_on_board`	Impediu a partida prematura do vagão enquanto havia pessoas caminhando para o assento
Manutenção da checagem de saída e `pthread_cond_signal` dentro da seção crítica em `station_on_board`	Garantir a leitura atômica dos contadores antes de acionar a variável de condição	Eliminou race conditions que faziam o vagão perder o sinal de partida e travar o sistema

5. Estratégias e Diário de Desenvolvimento

5.1 Estratégias da semana

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	Construção Incremental	Implementar as funções necessárias para fazer a sincronização do metrô aos poucos	

5.2 Diário de Tentativas

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	S1	Defini a Estrutura e a Inicialização (`struct station` e `station_init`)	Implementado		28/10	commit #1
2	S1	Defini a função de passageiro chegando e esperando na estação	Implementado		29/10	commit#2
3	S1	Implementação do embarque do passageiro	Implementado		29/10	commit#3
4	S1	Implementação da função station_load_car	Implementado		30/10	commit #4
5	S1	Criação do arquivo sync_car.h para usar nos testes	Implementado		30/10	commit #5

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
6	S1	Tentei rodar a primeira vez o teste passado	Falha	função load_car errada	30/10	evidencias.log
7	S1	Dentro da função load_car, acordei os passageiros usando `pthread_cond_broadcast`, e alterei o operador do loop while	Falha	função load_car errada	30/10	evidencias.log e print #1
8	S1	Mudei a localização do Unlock do Mutex em “station_on…”, alterei as duas condicionais do loop while	Funcionando		03/11	evidências.log

6. Evidências

print #1

```
davi@computadordavi:~/Implementacao4/dmcv$ ./sync_car
Error: Failed to complete after 1 seconds. Something's wrong, or your system is terribly slow. Possible error hint: [station_load_car() did not return immediately when no waiting passengers]
davi@computadordavi:~/Implementacao4/dmcv$ exit
```

7. Uso de IA

Onde usei IA:

Usei a IA(Gemini) como apoio no diagnóstico de problemas que foram ocorrendo ao longo do processo, ajudando a entender alguns cenários de race condition(foi especificado que o contexto desses cenários teria que ser totalmente diferente do da questão), além disso foi pedido explicações teóricas sobre as primitias “pthread_cond_t”.

Prompts principais:

“Não entendi muito bem o que as variáveis de condição (`pthread_cond_t`) fazem na prática, pode me explicar?”

“Analise a lógica do meu `while` na `station_load_car` para entender o motivo de estar dando timeout no teste do professor ”

“Você acha que eu deva me preocupar com o passageiro a caminho do vagão no meu código?”

O que validei manualmente e como:

Validei manualmente toda a lógica dos laços while e das seções críticas fazendo a checagem do fluxo de threads no papel. Verifiquei se o Makefile compilava sem *warnings* via GCC e executei os testes gravados no evidencias.log até confirmar que todas as asserções do runner passavam sem falhas

8. Reflexão Final:

Confesso que a lógica de concorrência com threads e primitivas POSIX foi bastante desafiadora, principalmente por exigir um raciocínio muito apurado sobre *race conditions* e estados intermediários. O que facilitou muito foi desenhar a simulação do fluxo de embarque, onde consegui visualizar inclusive algo que talvez me desse problema no futuro, que era verificar a quantidade de passageiros que ainda estavam embarcando. Além disso, entender que as variáveis de condição funcionam como salas de espera independentes foi o ponto de virada para organizar o fluxo entre passageiros e o vagão. O maior aprendizado foi perceber o impacto de liberar o mutex cedo demais em station_on_board e como uma condição de laço imprecisa pode travar o sistema inteiro em *deadlock*.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	OK
Rodei os testes e evidencias.log foi gerado	OK
Tenho prints obrigatórios	OK
Testei pelo menos um caso limite e um caso inválido	OK
Preenchi a seção de uso de IA	OK
Revisei o relatório e removi frases genéricas / vazias	OK

10. Se eu tivesse mais 2 horas

Se eu tivesse mais duas horas, aproveitaria para mexer no arquivo de testes (sync_car_runner.c) e colocar latências aleatórias na execução das threads para testar o comportamento sob estresse de hardware, assunto que estou vendo na cadeira de Infraestrutura de Hardware. Também daria uma revisada geral no código para simplificar as checagens dentro do mutex e deixar a estrutura da struct station e das funções ainda mais modulares e fáceis de dar manutenção